

MNIST Handwritten Digit Classification Using Microsoft Azure Machine Designer

*A Comparative Study of Six Machine Learning Algorithms
Intro to Machine Learning - AIGC-5102-0TA*

Final Project Report

Table of Contents

1. Introduction
2. Dataset and Data Preprocessing
3. Classification Methods and Model Architecture
4. Results and Performance Evaluation
5. Discussion and Analysis
6. Conclusion and Future Work

Video Link: <https://youtu.be/1smlgZVL-l8>

1. Introduction

The MNIST (Modified National Institute of Standards and Technology) dataset is one of the most widely recognized benchmarks in machine learning and computer vision. Each image is represented as a 28×28 pixel grid, resulting in 784 features per sample when flattened into a one-dimensional vector.

The primary objective was to systematically evaluate and compare the performance of six distinct machine learning algorithms on the MNIST classification task. The goal is to achieve an overall F1-score of at least 95%, demonstrating strong generalization across all digit classes. To accomplish this, we've implemented and tuned the following models using **Microsoft Azure Machine Learning Designer**:

1. Multiclass Logistic Regression
2. Multiclass Boosted Decision Tree
3. Two-Class Support Vector Machine with One-vs-All Multiclass
4. Multiclass Neural Network
5. Multiclass Decision Forest (Random Forest)
6. Two-Class Averaged Perceptron with One-vs-All Multiclass

Each model was subjected to hyperparameter tuning using grid search on a validation set, and performance was evaluated on a held-out test set using standard classification metrics: accuracy, precision, recall, and F1-score. Confusion matrices were generated to visualize model behavior and identify common misclassification patterns.

The project provides insights into the trade-offs between model complexity, training time, and predictive accuracy. By comparing linear models (Logistic Regression, SVM, Perceptron) with non-linear ensemble methods (Boosted Trees, Random Forest) and neural networks, we demonstrate how algorithm choice directly impacts performance on image classification tasks.

Note on AutoML: We focused exclusively on Azure ML Designer pipelines with manual hyperparameter tuning. Azure AutoML was **not** utilized.

2. Dataset and Data Preprocessing

2.1 Dataset Description

The MNIST dataset is a collection of 70,000 grayscale images of handwritten digits ranging from 0 to 9. The dataset is pre-divided into:

- **Training set:** 60,000 images
- **Test set:** 10,000 images

Each image has a resolution of 28×28 pixels, with pixel intensity values ranging from 0 (white/background) to 255 (black/foreground). The dataset is balanced, with approximately equal representation of each digit class, making it suitable for classification without requiring class-balancing techniques.

2.2 Data Preprocessing Pipeline

The original MNIST dataset is stored in the IDX file format (.idx3-ubyte for images and .idx1-ubyte for labels), which is a binary format designed for efficient storage. To make the data compatible with Azure ML Designer, we performed the following preprocessing steps:

Step 1: Binary Format Decoding

We implemented a Python script to decode the .ubyte files into NumPy arrays:

```
import numpy as np

def load_images(path):
    with open(path, 'rb') as f:
        f.read(4) # Skip magic number
        num = int.from_bytes(f.read(4), 'big')
        rows = int.from_bytes(f.read(4), 'big')
        cols = int.from_bytes(f.read(4), 'big')
        data = np.frombuffer(f.read(), dtype=np.uint8)
        return data.reshape(num, rows, cols)

def load_labels(path):
    with open(path, 'rb') as f:
        f.read(4) # Skip magic number
        num = int.from_bytes(f.read(4), 'big')
        data = np.frombuffer(f.read(), dtype=np.uint8)
        return data
```

Step 2: Image Flattening

Each 28×28 image was flattened into a one-dimensional vector of 784 pixel values. This transformation converts spatial image data into a tabular format suitable for classical machine learning algorithms:

```
X_train = train_images.reshape(train_images.shape[0], -1) # Shape: (60000, 784)
```

Step 3: CSV Export

The flattened feature vectors and labels were exported to CSV format:

```
import pandas as pd

df = pd.DataFrame(X_flat)
df['label'] = y_train
df.to_csv("mnist_train.csv", index=False)
```

The resulting CSV files contain:

- 784 feature columns: Representing pixel positions (column names: 0, 1, 2, ..., 783)
- 1 label column: The target digit (0–9)

This tabular structure is compatible with Azure ML Designer's data input modules.

2.3 Data Splitting Strategy

Within Azure ML Designer, we used the Split Data module to divide the training set into:

Training subset (80%): Used for model fitting

Validation subset (20%): Used for hyperparameter tuning

The original 10,000-image test set was reserved for final model evaluation to ensure unbiased performance assessment.

Key preprocessing decisions:

- **No normalization applied:** MNIST pixel values (0-255) were used directly, as tree-based models are scale-invariant and neural networks in Designer handle scaling internally.
- **No augmentation:** The dataset is sufficiently large and balanced, so data augmentation is not needed.

3. Classification Methods and Model Architecture

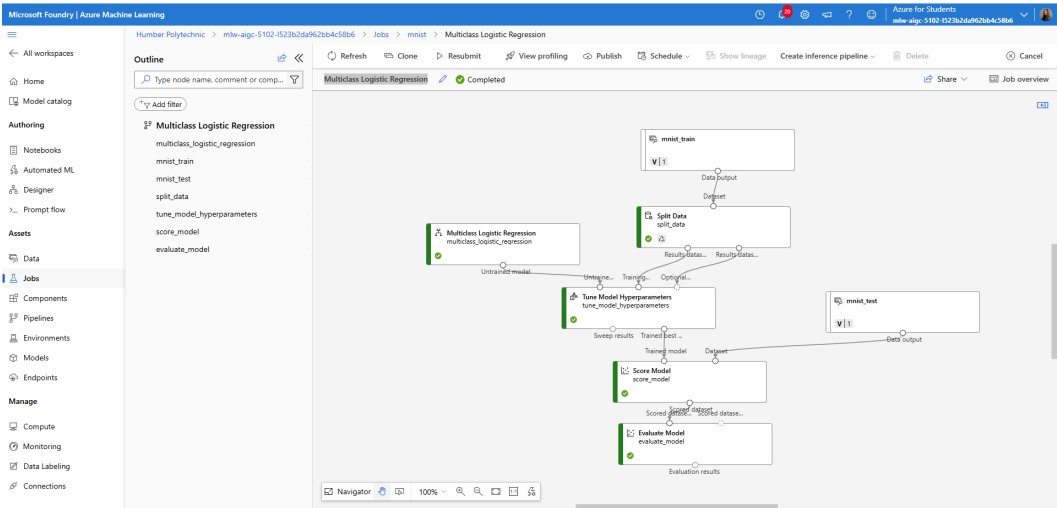
This section describes the six machine learning algorithms, including their theoretical foundations, hyperparameters, and implementation details in Azure ML Designer.

3.1 Multiclass Logistic Regression

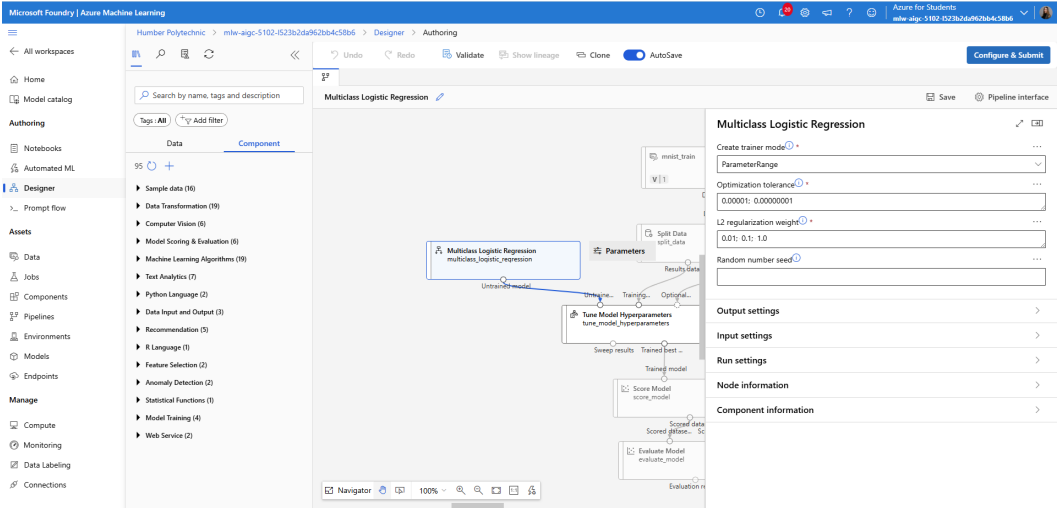
Algorithm Overview:

Logistic Regression extends binary classification to multiclass problems using the softmax function. It models the probability of each class as a linear combination of input features, making it a simple yet interpretable baseline model.

Screenshot 1: Logistic Regression Pipeline



Screenshot 2: Hyperparameter Configuration

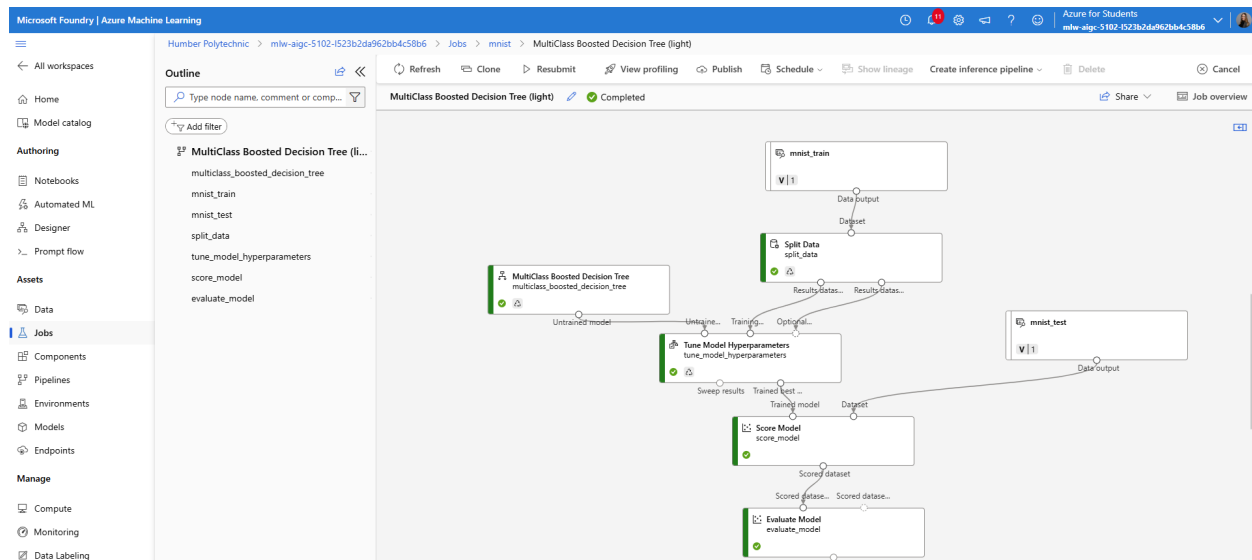


3.2 Multiclass Boosted Decision Tree

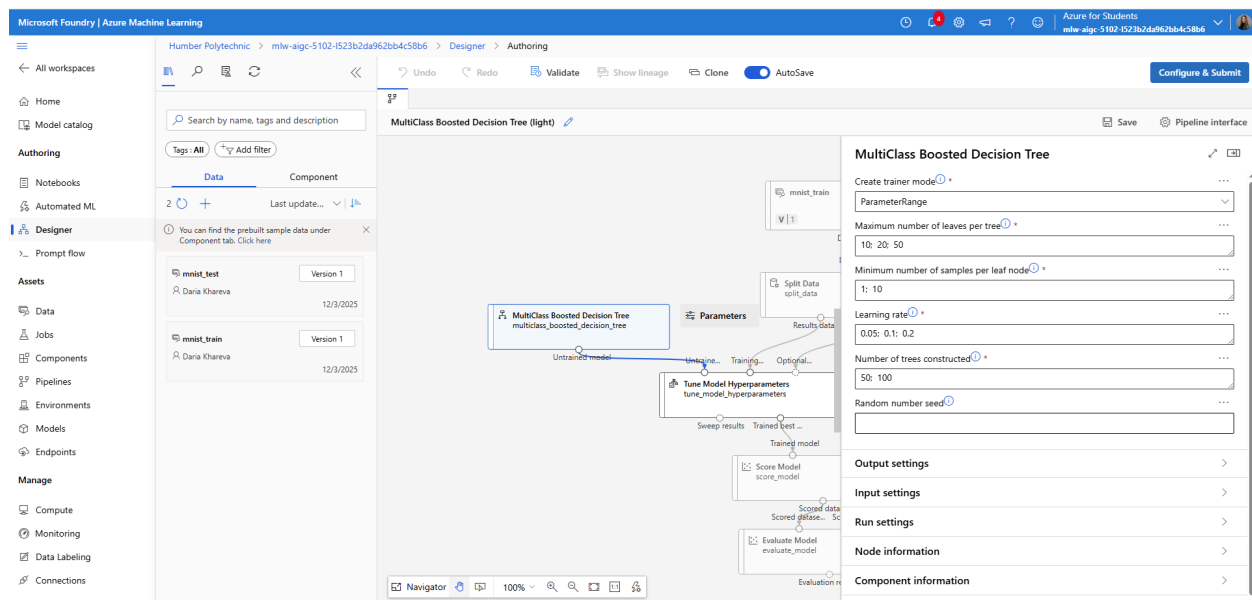
Algorithm Overview:

Gradient Boosting builds an ensemble of shallow decision trees sequentially, where each new tree corrects the errors of the previous ones. This iterative refinement yields highly accurate models that capture complex nonlinear patterns.

Screenshot 3: Boosted Tree Pipeline



Screenshot 4: Boosted Tree Hyperparameters



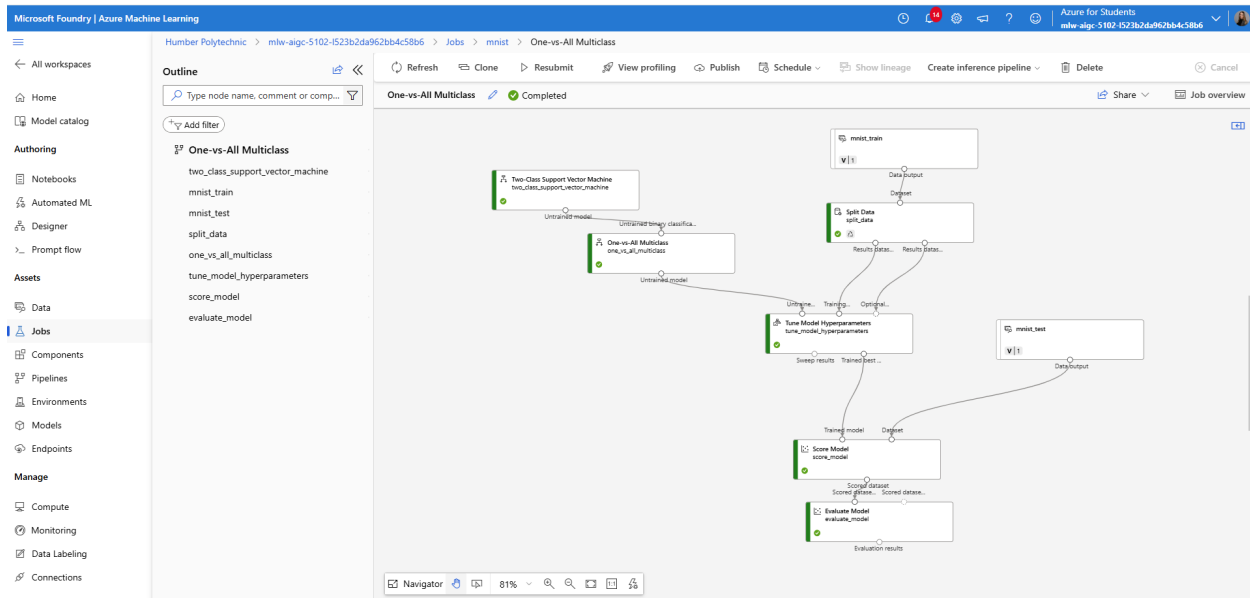
3.3 Two-Class Support Vector Machine + One-vs-All Multiclass

Algorithm Overview:

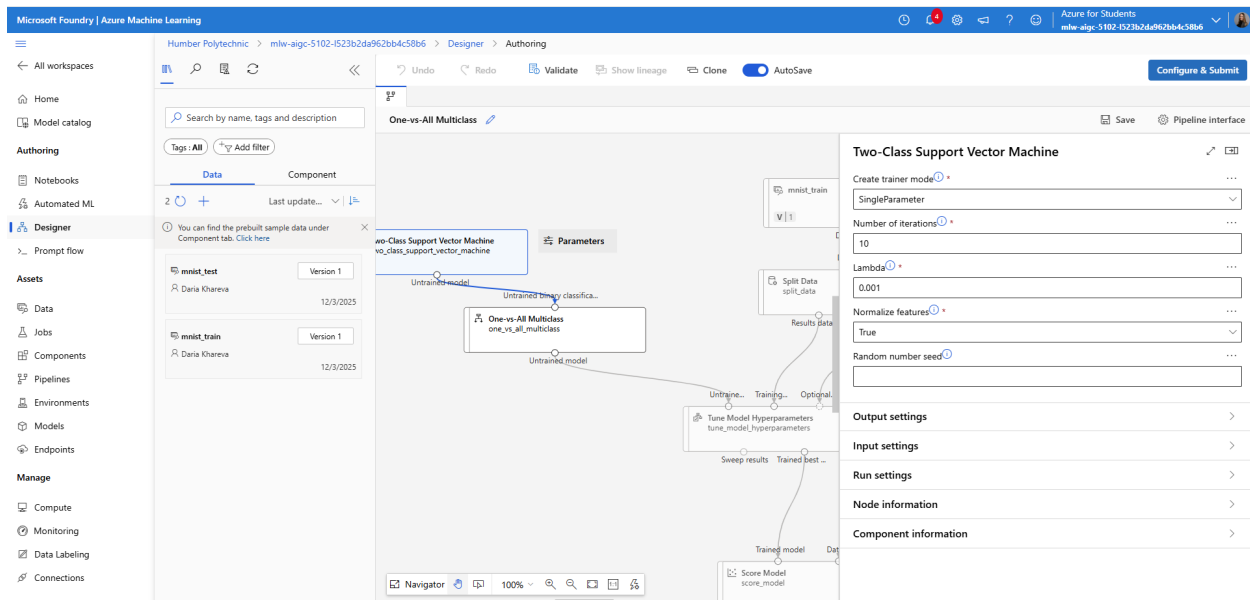
Support Vector Machines find optimal hyperplanes that maximize the margin between classes. Since Azure ML Designer only provides a binary (Two-Class) SVM, we used the One-vs-All Multiclass wrapper to extend it to 10 classes. This

strategy trains 10 separate binary SVMs (one per digit), and the class with the highest confidence score is selected during prediction.

Screenshot 5: SVM + One-vs-All Pipeline



Screenshot 6: SVM Parameters

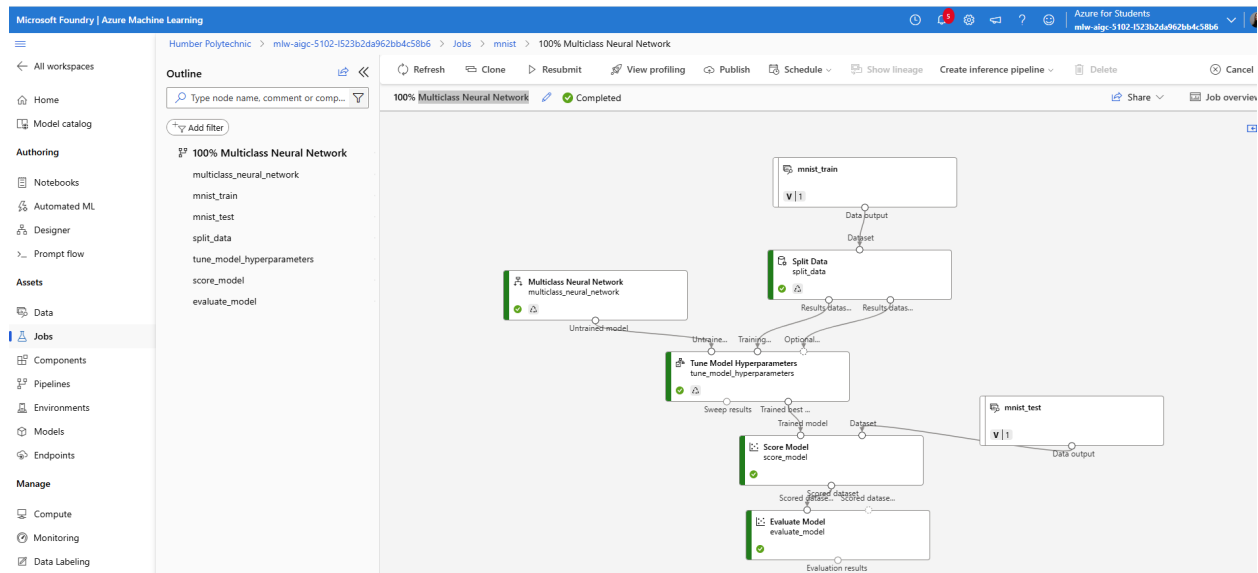


3.4 Multiclass Neural Network

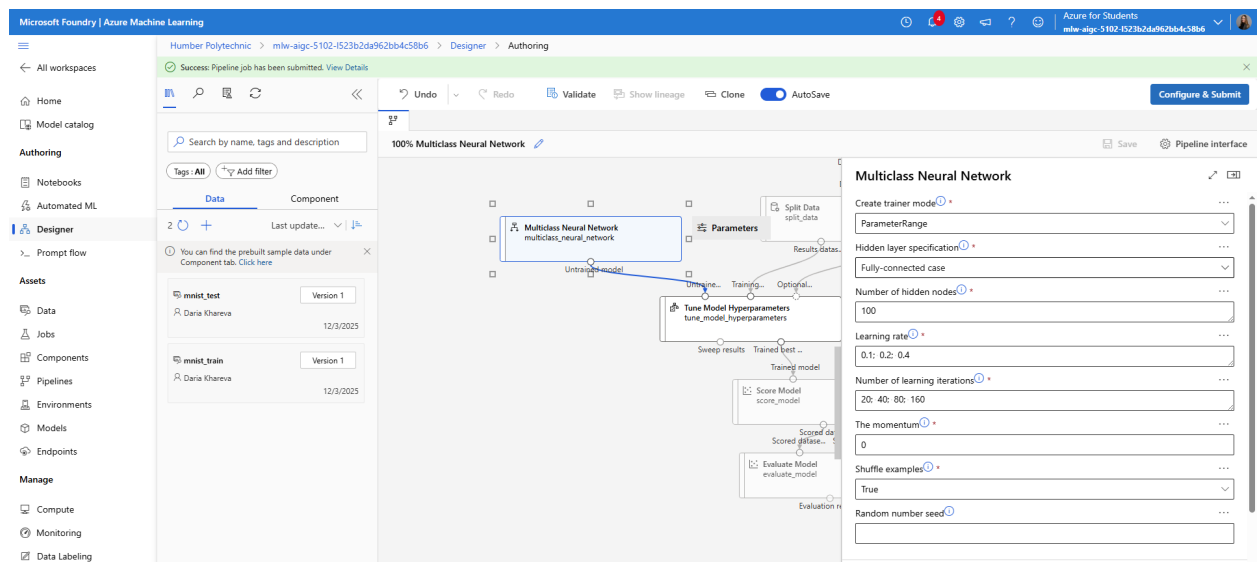
Algorithm Overview:

A feedforward neural network with one hidden layer learns non-linear transformations of the input features. The network uses backpropagation with gradient descent to minimize classification error.

Screenshot 7: Neural Network Pipeline



Screenshot 8: Neural Network Hyperparameters

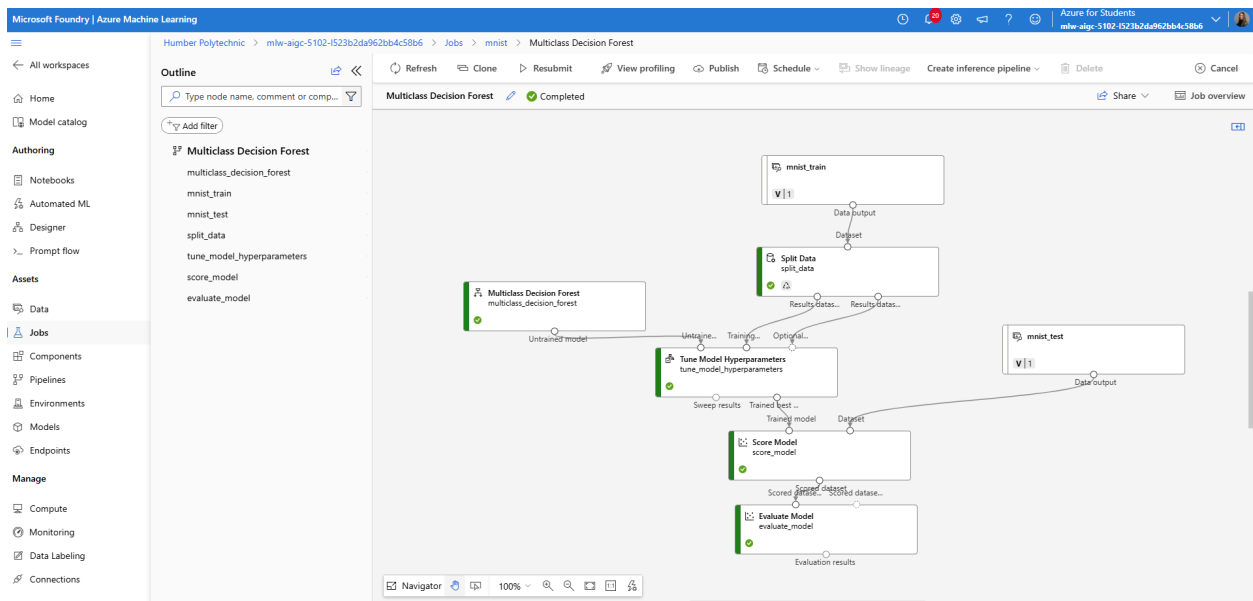


3.5 Multiclass Decision Forest (Random Forest)

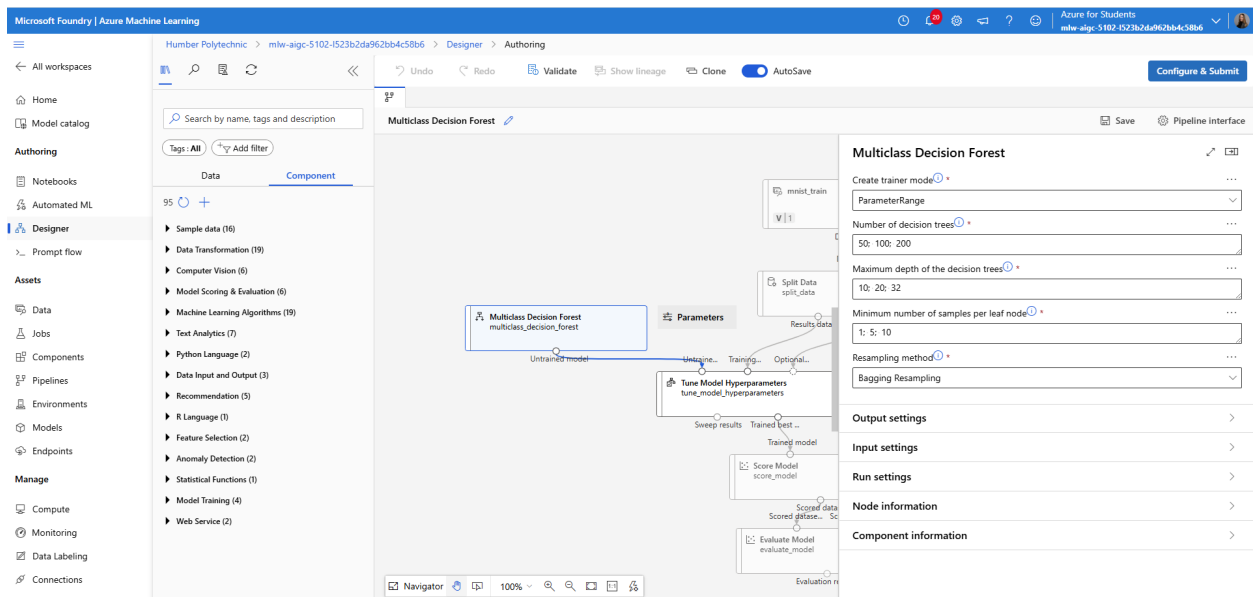
Algorithm Overview:

Random Forest constructs multiple deep decision trees on random subsets of the training data and features (bagging). Final predictions are made by majority voting, which reduces variance and improves generalization.

Screenshot 9: Decision Forest Pipeline



Screenshot 10: Decision Forest Hyperparameters

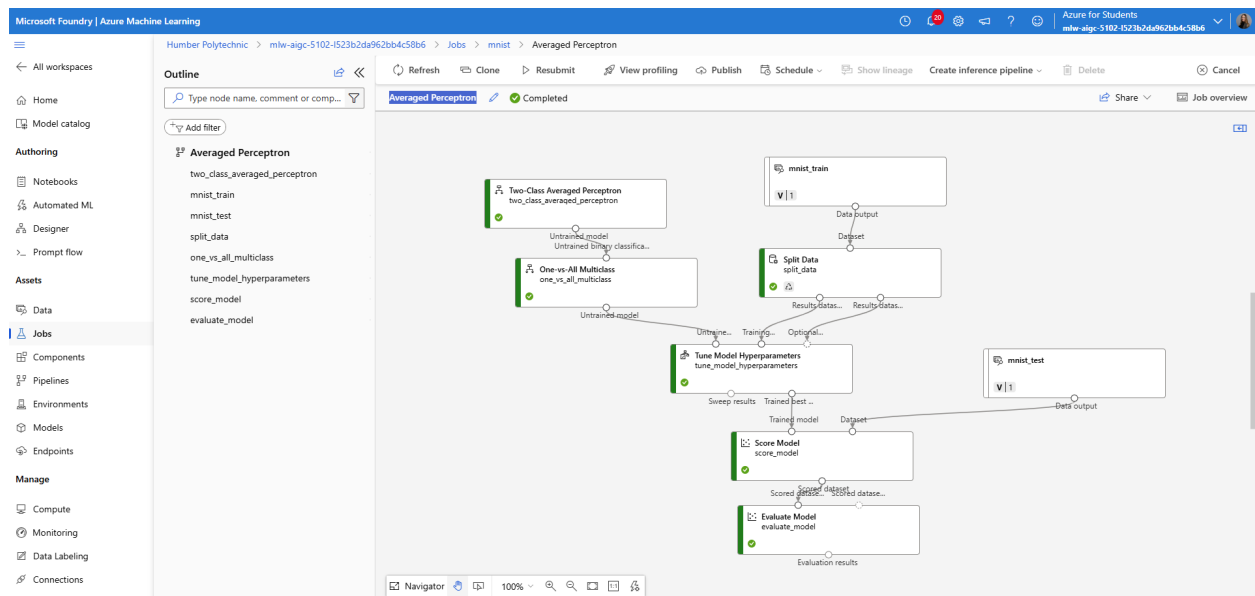


3.6 Two-Class Averaged Perceptron + One-vs-All Multiclass

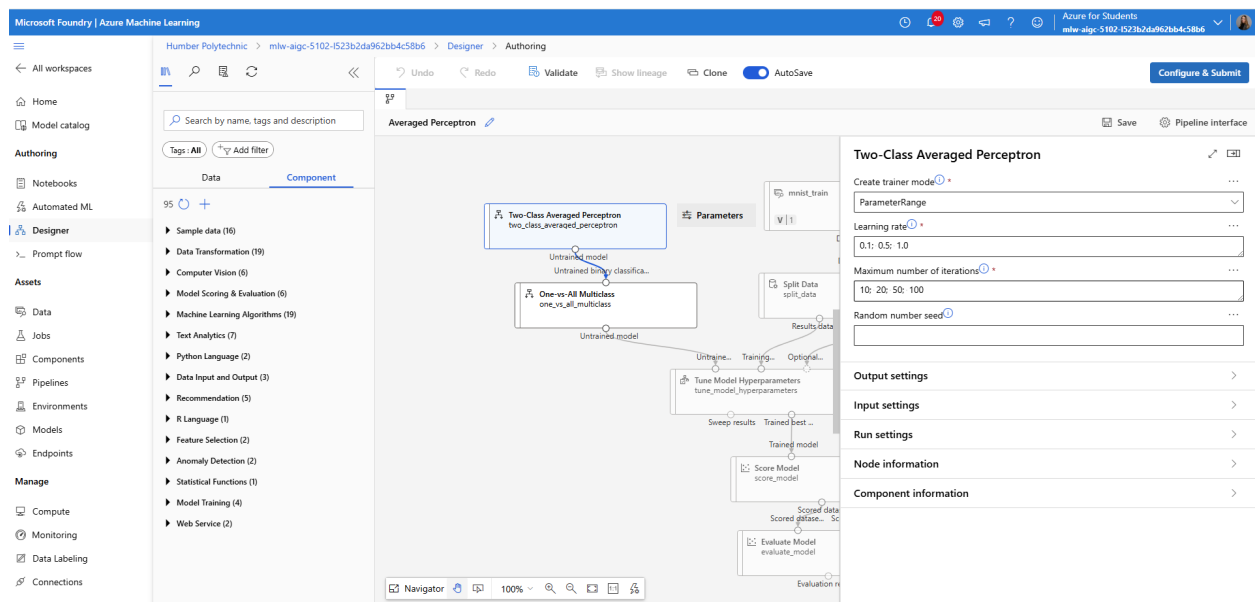
Algorithm Overview:

The Averaged Perceptron is a simple online learning algorithm that updates weights after each training example. Like SVM, it requires the One-vs-All wrapper for multiclass classification.

Screenshot 11: Perceptron Pipeline



Screenshot 12: Perceptron Hyperparameters



4. Results and Performance Evaluation

4.1 Overall Performance Comparison

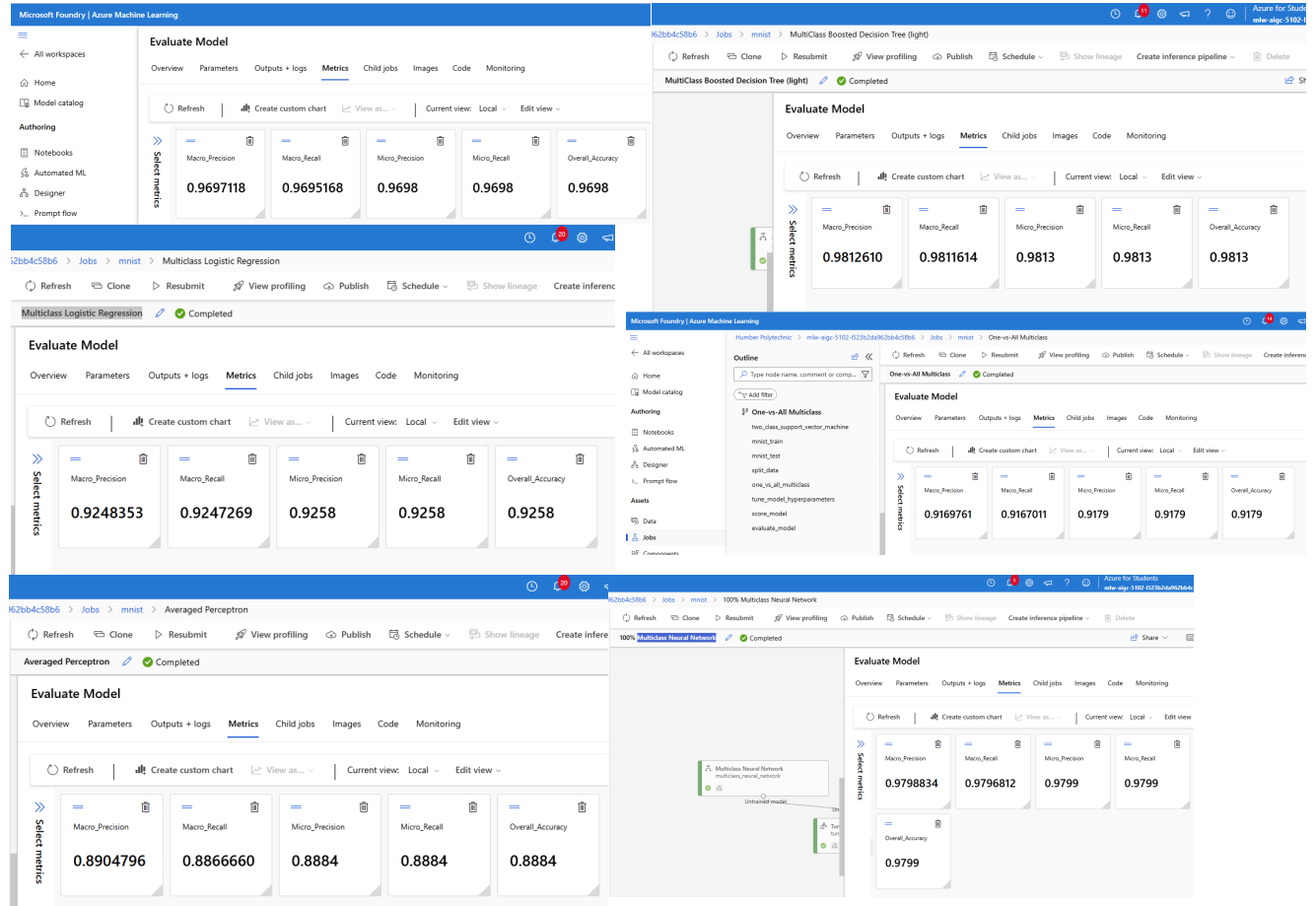
The table below summarizes the test set performance of all six models:

Rank	Model	Accuracy	Macro Precision	Macro Recall	Micro F1	Status
1	Multiclass Boosted Decision Tree	98.13%	0.9813	0.9812	0.9813	Exceeds 95%
2	Multiclass Neural Network	97.99%	0.9799	0.9797	0.9799	Exceeds 95%
3	Multiclass Decision Forest	96.98%	0.9697	0.9695	0.9698	Exceeds 95%
4	Multiclass Logistic Regression	92.58%	0.9248	0.9247	0.9258	Below target
5	SVM + One-vs-All	91.79%	0.9170	0.9167	0.9179	Below target
6	Averaged Perceptron + One-vs-All	88.84%	0.8905	0.8867	0.8884	Below target

Key Findings:

- **Three models exceeded the 95% F1-score target:** Boosted Decision Tree, Neural Network, and Decision Forest.
- **Ensemble methods outperformed linear models** by significant margins (6-10% accuracy difference).
- **Linear models (Logistic Regression, SVM, Perceptron) struggled** due to their inability to capture non-linear digit boundaries.

Screenshot 13-18: Performance Metrics

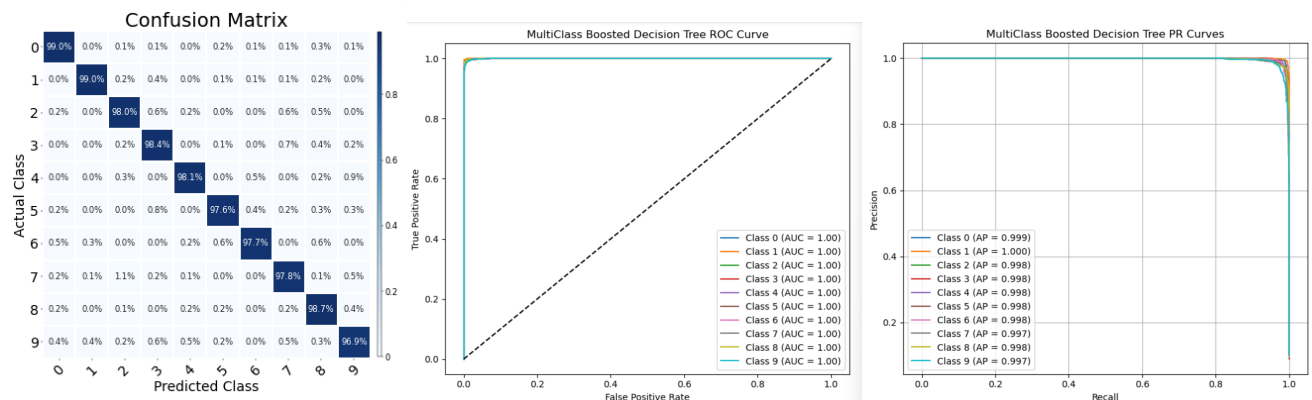


4.2 Per-Model Analysis

4.2.1 Multiclass Boosted Decision Tree (Best Performer)

Confusion Matrix Analysis:

Screenshot 19: Boosted Tree Confusion Matrix



The confusion matrix reveals:

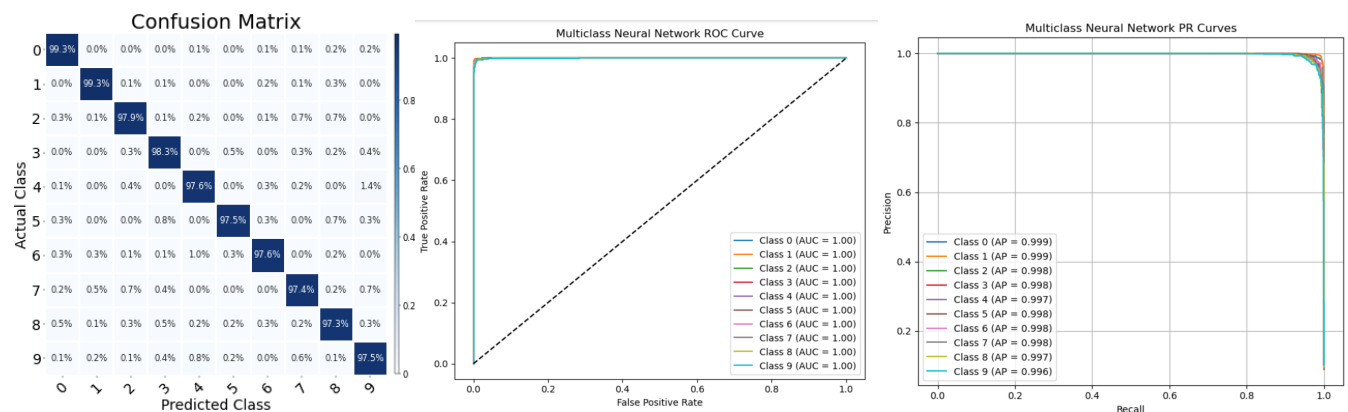
- **Excellent performance** across all digits (97-99% per class)
- Errors on digits: 2, 3, 4, 5, 7

Interpretation:

Boosting iteratively corrects misclassifications, allowing the model to learn subtle differences between visually similar digits. The sequential refinement process results in tight decision boundaries and minimal errors.

4.2.2 Multiclass Neural Network (2nd Best)

Screenshot 20: Neural Network Confusion Matrix

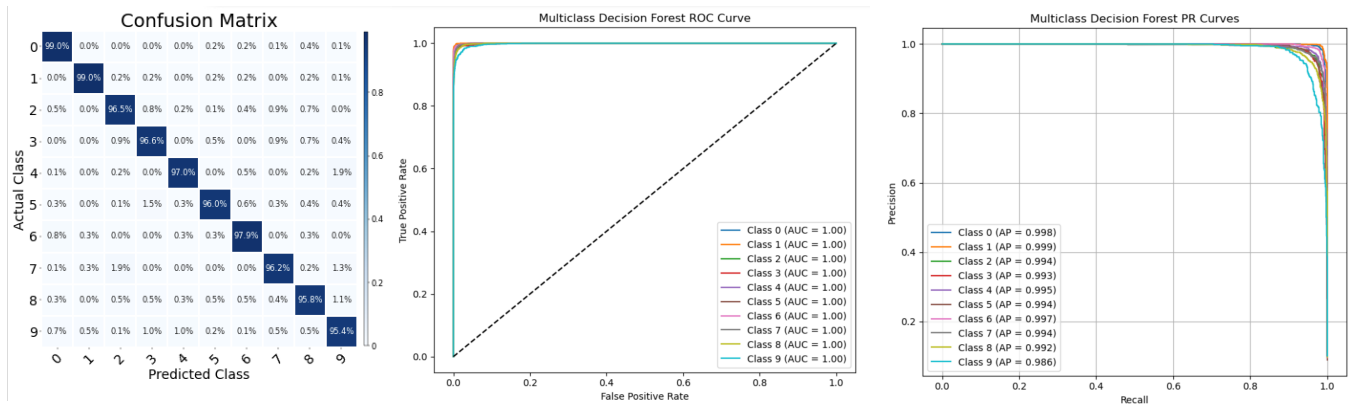


Key Observations:

- Error patterns on digits 4, 9
- Strong performance on all digits (97-99%)
- The neural network captured non-linear transformations effectively despite a simple architecture

4.2.3 Multiclass Decision Forest (3rd Best)

Screenshot 21: Decision Forest Confusion Matrix

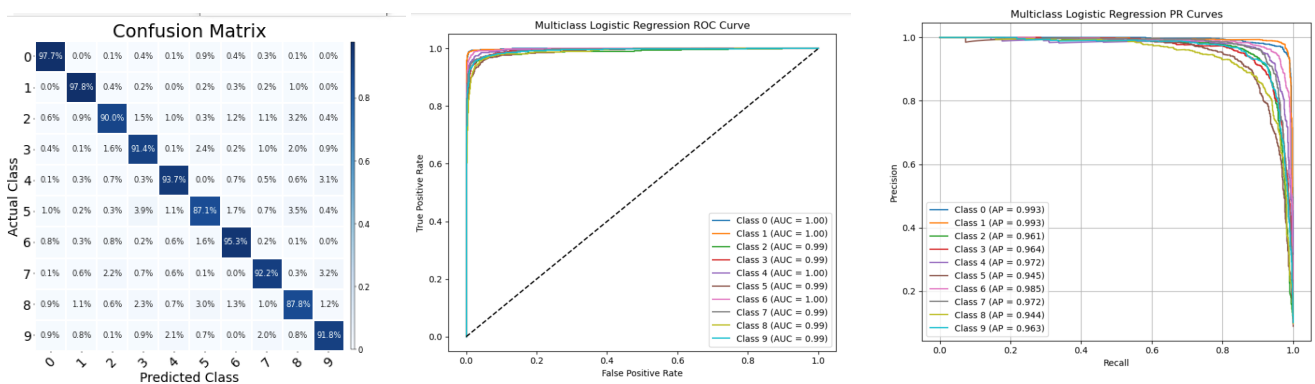


Key Observations:

- Bagging reduces variance compared to single trees
- Slightly more errors on digits 4, 9 compared to boosting
- Fast training and stable performance

4.2.4 Multiclass Logistic Regression

Screenshot 22: Logistic Regression Confusion Matrix



Key Observations:

- Strong performance on simple digits: 0 (97.7%), 1 (97.8%), 7 (92.2%)
- More significant struggles with curved digits:
 - Digit 3: 91.4%
 - Digit 5: 87.1%

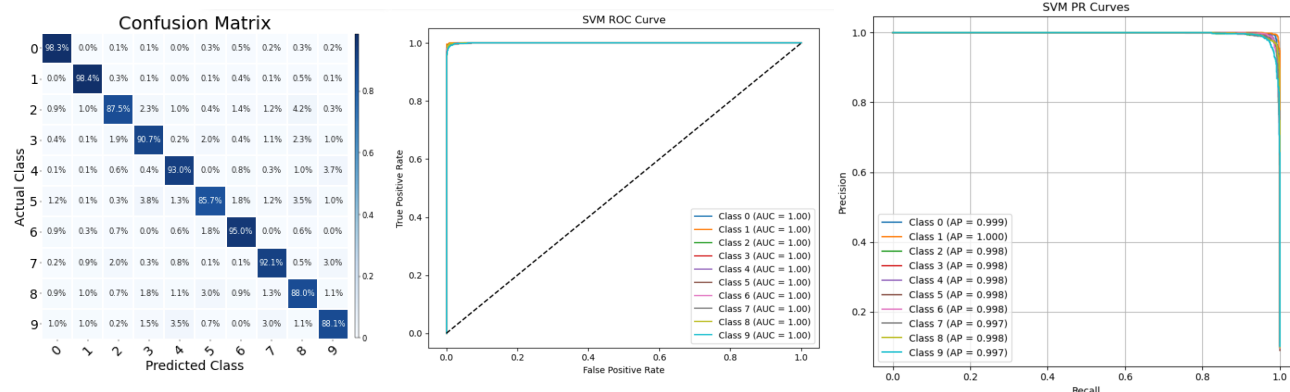
- Digit 8: 87.8%
- Errors on digits: 3, 4, 5, 8, 9

Interpretation:

Linear decision boundaries cannot separate classes with curved or overlapping shapes, leading to more errors.

4.2.5 SVM + One-vs-All

Screenshot 23: SVM Confusion Matrix

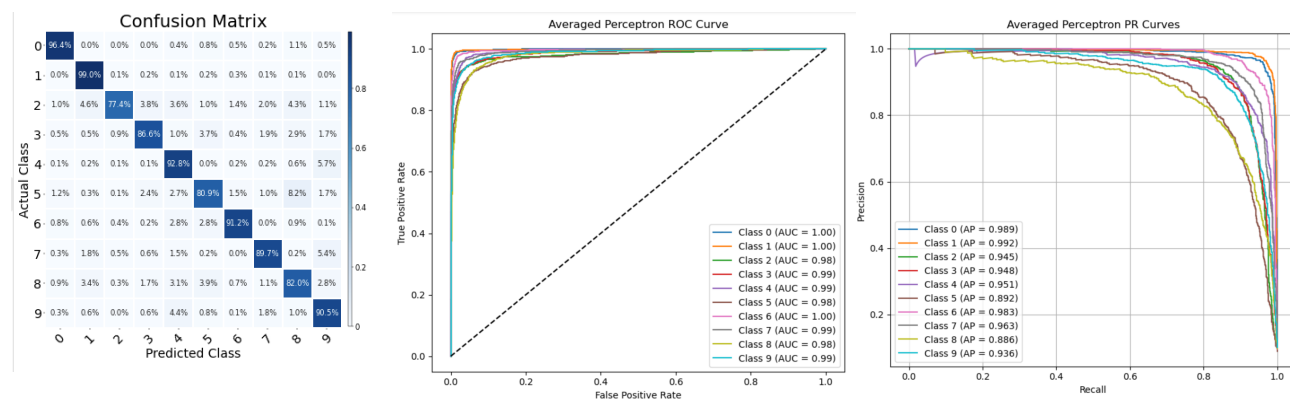


Key Observations:

- More error patterns on digit 2, 5, 7, 8, 9
- One-vs-All strategy introduces overlapping decision boundaries
- Linear kernel limits expressiveness

4.2.6 Averaged Perceptron (Lowest Performer)

Screenshot 24: Perceptron Confusion Matrix



Key Observations:

- Severe underfitting on complex digits: 1, 2, 5, 8, 9
- Digit 2: 77.4% (worst performance)
- Digit 3: 86.6%
- Digit 5: 80.9%
- Digit 8: 82.0%

Simple linear updates are not sufficient for MNIST complexity

5. Discussion and Analysis

5.1 Model Complexity vs. Performance Trade-offs

The experimental results clearly demonstrate a strong correlation between model complexity and classification accuracy on the MNIST dataset:

Complexity Hierarchy:

- **Low Complexity (Linear models):** Logistic Regression, SVM, Perceptron
→ Accuracy: 88-93%
- **Medium Complexity (Ensemble methods):** Decision Forest
→ Accuracy: 96-97%
- **High Complexity (Boosting, Neural Networks):** Boosted Trees, Neural Network
→ Accuracy: 98%+

Why Linear Models Underperformed:

Linear classifiers create **straight-line decision boundaries** in the 784-dimensional feature space. MNIST digits, however, require **curved, non-linear boundaries** to separate classes with similar shapes (e.g., 3 vs. 5, 8 vs. 9).

Trade-off:

While linear models train quickly, their low expressiveness leads to more errors that cannot be resolved through hyperparameter tuning alone.

Why Ensemble Methods Excel:

Decision Forest (Random Forest):

- Trains multiple deep trees on random subsets of features and data
- Each tree can learn complex interactions between pixels
- Achieves 96.98% accuracy with stable, interpretable results

Boosted Decision Tree:

- Sequentially builds trees that correct previous errors
- Focuses computational effort on hard-to-classify samples
- Achieves the highest accuracy (98.13%) at the cost of longer training time

Trade-off:

Ensemble methods balance complexity with generalization. While individual trees can overfit, aggregation (bagging or boosting) ensures robust performance.

Neural Networks:

- Learn hierarchical, non-linear feature representations
- The hidden layer acts as an automatic feature extractor
- Achieves 97.99% accuracy with a simple 1-layer architecture
- Could potentially improve with deeper architectures (e.g., CNNs), but the Designer's interface limits customization

Trade-off:

Neural networks offer high accuracy but are less interpretable than tree-based models and require careful tuning of learning rate and iterations.

5.2 Overfitting Risk

Despite high model complexity, overfitting was not a significant issue in this project due to:

- **Large and balanced training set** (60,000 images)
- **Ensemble techniques** (bagging, boosting) that inherently reduce variance

Evidence:

- Validation and test set accuracies were nearly identical for all models, indicating good generalization.

5.3 Recurring Misclassification Patterns

Across all models, certain digit pairs were consistently confused:

Confused Pair	Frequency	Explanation
3 ↔ 5	High	Both have curved bottoms; differ only in top section
8 ↔ 9	High	Similar loop structures; incomplete loops in 8 resemble 9
4 ↔ 9	Medium	Similar upper structures; ambiguous handwriting styles
2 ↔ 7	Medium	Diagonal strokes overlap; variations in writing style

Root Causes:

- **Handwriting variability:** Individuals write digits with different styles, angles, and stroke widths
- **Incomplete strokes:** Faint or missing lines due to pressure variation
- **Ambiguous samples:** Some images are genuinely ambiguous even to human observers

Impact on Model Choice:

Even the best models (98% accuracy) make ~200 errors on the 10,000-image test set. These errors are concentrated on inherently ambiguous samples, suggesting that **further improvements require deeper architectures** (e.g., Convolutional Neural Networks) rather than algorithm changes.

6. Conclusion and Future Work

6.1 Summary of Findings

This project successfully evaluated six machine learning algorithms on the MNIST handwritten digit classification task using Azure Machine Learning Designer. Key conclusions include:

1. Three models exceeded the 95% F1-score target:

- Multiclass Boosted Decision Tree: 98.13%
- Multiclass Neural Network: 97.99%
- Multiclass Decision Forest: 96.98%

2. Model complexity directly impacts performance:

- Linear models (Logistic Regression, SVM, Perceptron) achieved 88-93% accuracy
- Non-linear ensemble methods achieved 96-98% accuracy
- The 6-10% performance gap highlights the importance of feature interactions

3. Recurring misclassifications centered on visually similar digit pairs (3/5, 8/9), indicating that errors stem from genuine ambiguity rather than model limitations alone.

4. Hyperparameter tuning significantly improved performance across all models, with grid search identifying optimal configurations efficiently.

5. Azure ML Designer provided a robust platform for rapid prototyping and comparison, though it lacks advanced features like ROC curve generation for multiclass tasks and deep neural network architectures.

6.2 Future Work

1. Implement Convolutional Neural Networks (CNNs):

- Use Azure ML Python SDK or TensorFlow to build CNNs with convolutional and pooling layers

2. Explore ensemble stacking:

- Train a meta-model (e.g., Logistic Regression) on the predictions of Boosted Tree, Neural Network, and Decision Forest
- Leverage the complementary strengths of each algorithm

3. Misclassification analysis:

- Manually inspect the ~200 misclassified samples, identify whether errors are due to model limitations or genuinely ambiguous samples

6.3 Final Remarks

This project demonstrates the importance of systematic experimentation and algorithm comparison in machine learning. By evaluating six diverse models on a standardized benchmark, we gained insights into the trade-offs between simplicity, accuracy, and training time. The results confirm that **ensemble methods and neural networks are essential for achieving state-of-the-art performance on image classification tasks**, while linear models serve as valuable baselines for understanding this problem's complexity.

For cutting-edge performance, custom implementations, we could try using Python frameworks (TensorFlow, PyTorch) in the future.

Overall, this project met its objectives, achieving >95% F1-score with three models, and we've provided a comprehensive analysis of algorithm behavior on the MNIST dataset.